Godot Responsive Display & Multi-Device Input

Implementation guide for a single 2D RPG targeting PC, mobile, and controllers

Overview

Godot can adapt one game to different screen sizes, aspect ratios, and input devices. The recommended approach is to keep gameplay and combat logic shared while adding a small platform-support layer for display, input, and HUD presentation.

> **Core principle:** fitting the screen should not mean stretching the game. Preserve proportions, anchor the UI correctly, and decide deliberately how much extra world area the camera reveals.

1. Responsive display configuration

Use these as baseline settings in Project > Project Settings > Display > Window. Confirm the exact property names in your Godot 4.7 project.

Setting	Suggested baseline	Purpose
Viewport width / height	1280 × 720	A 16:9 design reference; not a requirement that every device use this physical resolution.
Stretch mode	canvas_items	Scales 2D canvas items with the window.
Stretch aspect	expand	Allows the visible viewport to expand for different aspect ratios rather than simply cropping to a fixed shape.
Orientation	Landscape, if intended	Use a consistent orientation if the game design is built around landscape play.

Expected behavior

• **16:9 phone:** use the normal landscape view.

• **Tall or portrait window:** the viewport may reveal more world vertically; ensure the UI and encounter design remain usable.

• **Ultrawide display:** more world may be visible horizontally.

• **Resizable PC window:** the game and anchored UI should adjust as the window changes.

2. Camera and world visibility

Keep the existing player camera zoom as the starting point. With an expanding viewport, different aspect ratios can reveal additional world area without changing zoom. Do not automatically zoom the camera just to fill the screen unless that behavior is a deliberate design choice.

Test camera limits, level boundaries, enemy visibility, and encounter pacing at each target aspect ratio. A taller view should not expose unfinished areas or allow players to see beyond intended level boundaries.

3. Use one shared InputMap

Gameplay code should react to named actions rather than hard-coded device buttons. Configure multiple bindings for the same action in Project Settings > Input Map.

Action	Mobile	Keyboard / mouse	Controller
move	Virtual joystick	WASD	Left stick / D-pad
basic_attack	Attack button	Mouse button	Trigger / face button
heavy_attack	Hold attack button	Hold assigned input	Hold assigned button
aim	Drag / aim control	Mouse position	Right stick
skill_1..skill_3	On-screen skill buttons	Assigned keys/buttons	Assigned buttons
dodge	On-screen button	Assigned key	Assigned button

These are example bindings, not mandatory defaults. Choose final bindings to fit the combat design. Mobile buttons should call the same gameplay actions or shared gameplay methods as keyboard and controller input, rather than implementing a separate combat system.

4. Platform-aware HUD and controls

Keep device-specific controls in separate UI groups within the HUD, for example:

• **MobileControls:** virtual joystick, attack, dodge, and skill buttons.

• **DesktopControls:** optional keyboard/mouse prompts; no touchscreen overlay.

• **Controller support:** gamepad input and optional controller-specific button prompts.

Suggested behavior

• On Android/iOS, show touch controls by default.

• On desktop, hide touch controls by default.

• Detect controller availability, but do not assume a connected controller is always the active device.

• Optionally track the last-used device and update control prompts when the player switches between keyboard/mouse and controller.

• Keep layout responsive using anchors and containers; avoid relying on fixed pixel positions for all screen sizes.

5. Fit with the existing RPG architecture

Layer	Responsibilities
Platform / presentation	Display settings, input-mode manager, responsive HUD, mobile controls.
Input	InputMap actions, device bindings, movement and aim input.
Gameplay	Player, AttackComponent2D, weapon definitions, SkillComponent2D, AI, and combat execution.

Keep platform-specific behavior out of weapon definitions and combat execution wherever possible. Bow, sword, attacks, and skills should work across devices through the shared input layer.

6. Recommended implementation sequence

1. Configure responsive viewport and stretch settings.

2. Test camera visibility and world boundaries at multiple aspect ratios without changing current camera zoom.

3. Standardize InputMap actions for movement, aiming, basic/heavy attacks, dodge, and skills.

4. Create mobile HUD controls and connect them to the shared actions.

5. Add controller bindings and optional active-device detection.

6. Test combat on PC, touch devices, and controller, including bow aiming, heavy-attack hold timing, and skill activation.

7. Validation checklist

■ Landscape 16:9 phone or emulator

■ Portrait / tall aspect ratio, if supported

■ Resizable desktop window

■ Ultrawide display or equivalent test viewport

■ Mobile touch controls appear only when intended

■ Keyboard/mouse and controller trigger the same gameplay actions

■ UI stays readable and avoids covering important combat information

■ Camera limits, enemy visibility, and level boundaries remain correct

■ Bow aiming, charged attacks, dodge, and all skill slots work on supported devices

Official reference

Godot 4.7 documentation — Multiple resolutions:
https://docs.godotengine.org/en/4.7/tutorials/rendering/multiple_resolutions.html

> Recommendation: implement this as a platform/input enhancement around the existing project. Start with display and InputMap foundations, then add the responsive HUD and controller support; avoid duplicating combat logic.